

云原生训练营

模块九-生产化集群的管理

孟凡杰

前 eBay 资深架构师



目录

- 计算节点相关
- 操作系统选择
- 节点资源管理
- 节点异常检测
- 常用节点问题排查手段
- 基于extended resource扩展节点资源
- 构建和管理高可用集群
- Cluster Autoscaler
- 集群管理实践案例分享
- 多租户集群管理

1. 计算节点相关

生产化集群的考量

计算节点：

- 如何批量安装和升级计算节点的操作系统？
- 如何管理配置计算节点的网络信息？
- 如何管理不同SKU（Stock Keeping Unit）的计算节点？
- 如何快速下架故障的计算节点？
- 如何快速扩缩集群的规模？

控制平面：

- 如何在主节点上下载、安装和升级控制平面组件及其所需的配置文件？
- 如何确保集群所需的其他插件，例如CoreDNS、监控系统等部署完成？
- 如何准备控制平面组件的各种安全证书？
- 如何快速升级或回滚控制平面组件的版本？

2. 操作系统选择

操作系统的评估与选择

- 通用操作系统

- Ubuntu
- CentOS
- Fedora



- 专为容器优化的操作系统

- 最小化操作系统

- CoreOS
- RedHat Atomic
- Snappy Ubuntu Core
- RancherOS



操作系统的评估与选择

- 操作系统评估和选型的标准
 - 是否有生态系统
 - 成熟度
 - 内核版本
 - 对运行时的支持
 - Init System
 - 包管理和系统升级
 - 安全

生态系统与成熟度

- 容器优化操作系统的优势
 - 小
 - 原子级升级和回退
 - 更高的安全性

	CentOS	Ubuntu	CoreOS	Atomic*	Snappy	RancherOS
生态系统和成熟度	通用操作系统		容器优化			
	成熟		最早的容器优化操作系统, 不过公司体量小, 目前已被收购。	Red Hat出品, 品质保证	Canonical 出品, 最初为移动设备设计	相对较新, RancherOS中运行的所有服务都是docker 容器。

云原生的原则

- 可变基础设施的风险
 - 在灾难发生的时候，难以重新构建服务。持续过多的手工操作，缺乏记录，会导致很难由标准初始化后的服务器来重新构建起等效的服务。
 - 在服务运行过程中，持续的修改服务器，就犹如程序中的可变变量的值发生变化而引入的状态不一致的并发风险。这些对于服务器的修改，同样会引入中间状态，从而导致不可预知的问题。
- 不可变基础设施（immutable infrastructure）
 - 不可变的容器镜像
 - 不可变的主机操作系统

Atomic

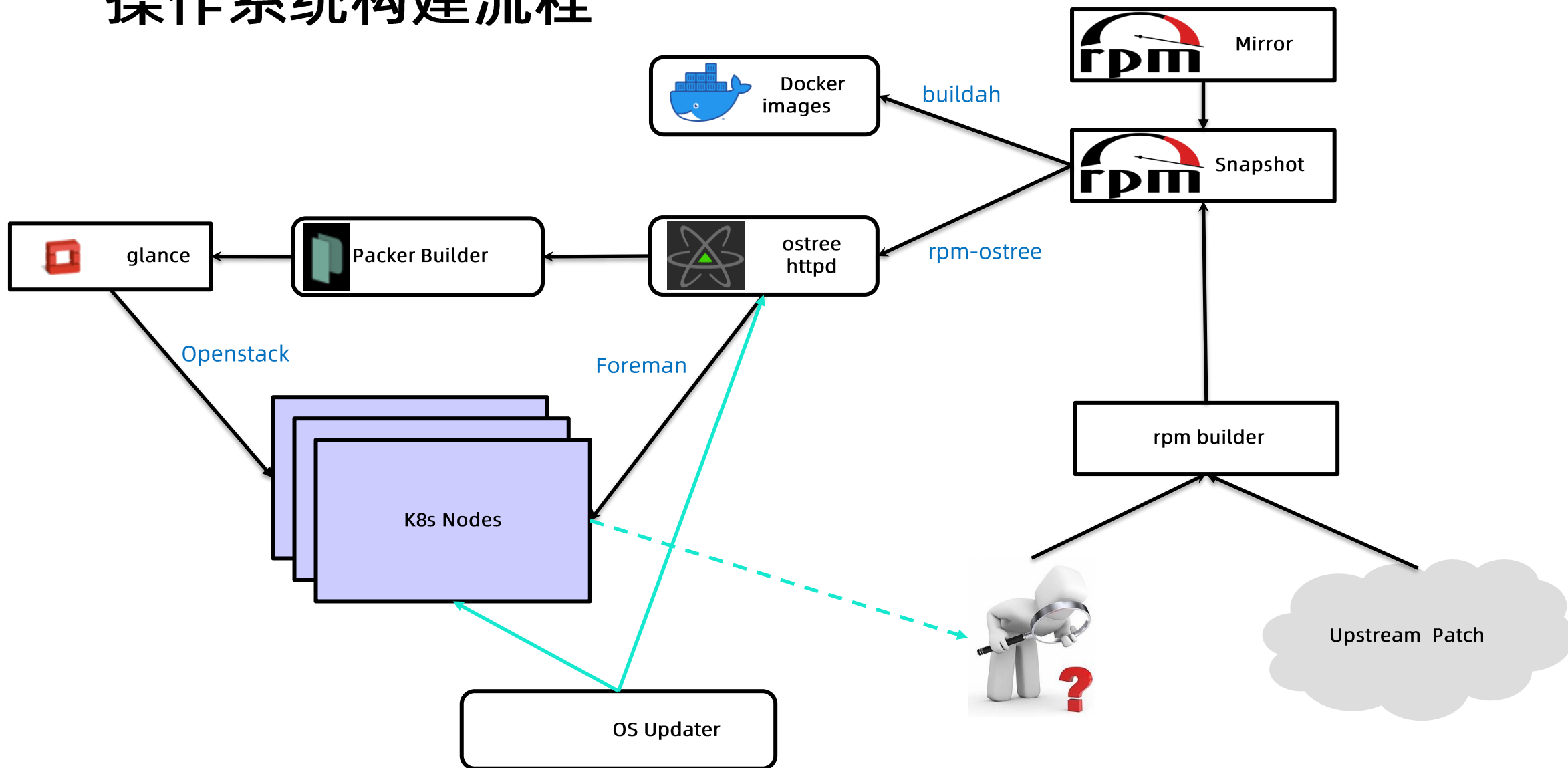
- 由Red Hat支持的软件包安装系统
- 多种Distro
 - Fedora
 - CentOS
 - RHEL
- 优势
 - 不可变操作系统，面向容器优化的基础设施
 - 灵活和安全性较好
 - 只有/etc和 /var可以修改，其他目录均为只读
 - 基于rpm-ostree管理系统包
 - rpm-ostree是一个开源项目，使得生产系统中构建镜像非常简单
 - 支持操作系统升级和回滚的原子操作

最小化主机操作系统

原则：

- 最小化主机操作系统
- 只安装必要的工具
 - 必要：支持系统运行的最小工具集
 - 任何调试工具，比如性能排查，网络排查工具，均可以后期以容器形式运行
- 意义
 - 性能
 - 稳定性
 - 安全保障

操作系统构建流程



ostree

提供一个共享库（libostree）和一些列命令行

提供与git命令行一致的体验，可以提交或者下载一个完整的可启动的文件系统树

提供将ostree部署进bootloader的机制

<https://github.com/ostreedev/ostree/blob/main/src/boot/dracut/module-setup.sh>

```
install() {  
    dracut_install /usr/lib/ostree/ostree-prepare-root  
    inst_simple "${systemdsystemunitdir}/ostree-prepare-root.service"  
    mkdir -p "${initdir}${systemdsystemconfdir}/initrd-root-fs.target.wants"  
    ln_r "${systemdsystemunitdir}/ostree-prepare-root.service" \  
        "${systemdsystemconfdir}/initrd-root-fs.target.wants/ostree-prepare-root.service"  
}
```

构建ostree

- rpm-ostree

基于treefile将rpm包构建成为ostree

管理ostree以及bootloader配置

- treefile

refer: 分支名 (版本, cpu架构)

repo: rpm package repositories

packages: 待安装组件

- 将rpm构建成为ostree

rpm-ostree compose tree --unified-core --
cachedir=cache --repo=./build-repo
/path/to/treefile.json

```
"osname": "centos-atomic-host",  
"ref": "centos-atomic-host/8/x86_64/standard",
```

```
"repos": [  
  "base",  
  "appstream",  
  "epel",  
  "docker-ce",  
  "ceph",  
  "salt",  
  "other",  
  "kernel-ml",  
  "ovs",  
  "zfs"  
],
```

```
"selinux": true,
```

```
"install-langs": ["en_US"],  
"documentation": false,
```

```
"initramfs-args": ["--no-hostonly"],  
"postprocess-script": "customize.sh",
```

```
"etc-group-members": ["wheel", "docker"],  
"ignore-removed-users": ["root"],  
"ignore-removed-groups": ["root"],  
"check-passwd": { "type": "file", "filename": "passwd" },  
"check-groups": { "type": "file", "filename": "group" },
```

```
"packages": [  
  "authconfig",  
  "ceph-common",  
  "ceph-fuse",  
  "chrony",  
  "cronie",  
  
  "cloud-init",  
  "cloud-utils-growpart",  
  "coreutils",  
  "conntack-tools",  
  "docker-ce",
```

加载ostree

- 初始化项目

ostree admin os-init centos-atomic-host

- 导入ostree repo

ostree remote add atomic <http://ostree.svr/ostree>

- 拉取ostree

ostree pull atomic centos-atomic-host/8/x86_64/standard

- 部署os

ostree admin deploy --os=centos-atomic-host centos-atomic-host/8/x86_64/standard --karg='root=/dev/atomicos/root'

```
[root@centos8-novgs-4139235 ostree]# tree .
.
├── config
├── extensions
├── objects
│   ├── 00
│   │   ├── 0002f94b3c712d49a33029e2809049d7d947dc659792cdc458ec5994e79460.filez
│   │   ├── 005d79aea748656db25d662f0f68faeda2a86618748942cee977b180a3ec57.filez
│   │   └── fe4306c7849d5f5f16ad9c0d8e30c7454f27e659cadd731d7377d7bd8926b3.filez
│   └── ...
│       ├── ca
│       │   └── 008300f95e1f8fa601313a417c0bf4d73b401bf36ddc45402bd8af5605ba3b.filez
│       └── ff
│           ├── 002c2fc6f675121779202534545ec19276f915e2cb6bf1650cbf98d66d460f.filez
│           └── ff2f5646515681f3ee781219dd958771ec8d557e2b9cc3ff47a5bec01dd17c.filez
├── refs
│   ├── heads
│   │   ├── centos-atomic-host
│   │   │   └── 8
│   │   │       └── x86_64
│   │   │           └── standard
│   ├── mirrors
│   └── remotes
├── state
├── summary
└── tmp
```

操作系统加载

- 物理机
 - 物理机通常需要通过foreman启动, foreman通过pxe boot, 并加载kickstart
 - kickstart通过ostree deploy即可完成操作系统的部署
- 虚拟机
 - 需要通过镜像工具将ostree构建为qcow2格式, vhd, raw等模式

生产环境遭遇过的陷阱

cloud-init 0.7.7 bug

- 阻止node在初始化过程中的静态网络配置

Docker 1.9.1 bug

- 当docker实例日志快速输出时，会发生内存泄漏

Kernel panic in 4.4.6

- Cgroup创建和销毁过程中，会产生kernel panic

rootfs 分区太小

- rootfs无空间会导致docker无法启动
 - 导致rootfs占满的情况
 - CI/CD 中的Maven build会把下载的lib放在 /tmp
 - 用户Docker logs 日志过快，导致一个log rotation周期内日志文件撑爆硬盘

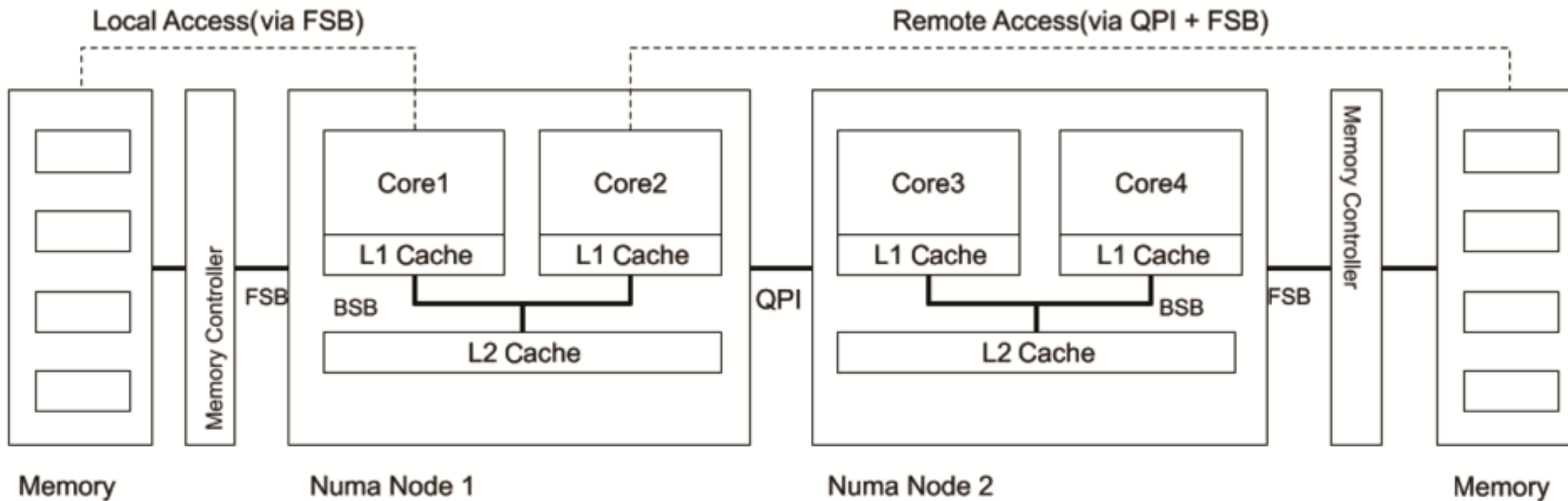
需要定制化的操作系统参数

- 比如Elasticsearch需要`max_map_count` ≥ 262144 ，但操作系统默认值为65535，我们需要在创建Node的时候就apply这个配置

3. 节点资源管理

NUMA Node

Non-Uniform Memory Access是一种内存访问方式，是为多处理器计算机设计的内存架构



节点资源管理

- 状态汇报
- 资源预留
- 防止节点资源耗尽的防御机制驱逐
- 容器和系统资源的配置

状态上报

kubelet周期性地向API Server进行汇报，并更新节点的相关健康和资源使用信息

- 节点基础信息，包括IP地址、操作系统、内核、运行时、kubelet、kube-proxy版本信息。
- 节点资源信息包括CPU、内存、HugePage、临时存储、GPU等注册设备，以及这些资源中可以分配给容器使用的部分。
- 调度器在为Pod选择节点时会将机器的状态信息作为依据。

状态	状态的意义
Ready	节点是否健康
MemoryPressure	节点是否存在内存压力
PIDPressure	节点是否存在比较多的进程
DiskPressure	节点是否存在磁盘压力
NetworkUnavailable	节点网络配置是否正确

Lease

在早期版本kubelet的状态上报直接更新node对象，而上报的信息包含状态信息和资源信息，因此需要传输的数据包较大，给APIServer和etcd造成的压力较大。

后引入Lease对象用来保存健康信息，在默认40s的nodeLeaseDurationSeconds周期内，若Lease对象没有被更新，则对应节点可以被判定为不健康。

```
apiVersion: coordination.k8s.io/v1
kind: Lease
metadata:
  creationTimestamp: "2021-08-19T02:50:09Z"
  name: k8snode
  namespace: kube-node-lease
  ownerReferences:
    - apiVersion: v1
      kind: Node
      name: k8snode
      uid: 58679942-e2dd-4ead-aada-385f099d5f56
  resourceVersion: "1293702"
  uid: 1bf51951-b832-49da-8708-4b224b1ec3ed
spec:
  holderIdentity: k8snode
  leaseDurationSeconds: 40
  renewTime: "2021-09-08T01:34:16.489589Z"
```

资源预留

计算节点除用户容器外，还存在很多支撑系统运行的基础服务，譬如systemd、journald、sshd、dockerd、Containerd、kubelet等。

为了使服务进程能够正常运行，要确保它们在任何时候都可以获取足够的系统资源，所以我们要为这些系统进程预留资源。

kubelet可以通过众多启动参数为系统预留CPU、内存、PID等资源，比如SystemReserved、KubeReserved等。

Capacity和Allocatable

容量资源（Capacity）是指kubelet获取的计算节点当前的资源信息。

- CPU是从/proc/cpuinfo文件中获取的节点CPU核数；
- memory是从/proc/memoryinfo中获取的节点内存大小；
- ephemeral-storage是指节点根分区的大小。

资源可分配额（Allocatable）是用户Pod可用的资源，是资源容量减去分配给系统的资源的剩余部分。

```
allocatable:
  cpu: "24"
  ephemeral-storage: 205838Mi
  memory: 177304536Ki
  pods: "110"
capacity:
  cpu: "24"
  ephemeral-storage: 205838Mi
  memory: 179504088Ki
  pods: "110"
```

节点的资源容量(Capacity)			
kube-reserved	system-reserved	eviction-threshold	Allocatable

节点磁盘管理

- 系统分区 nodefs
 - 工作目录和容器日志
- 容器运行时分区 imagefs
 - 用户镜像和容器可写层
 - 容器运行时分区是可选的，可以合并到系统分区中

驱逐管理

- kubelet会在系统资源不够时中止一些容器进程，以空出系统资源，保证节点的稳定性。
- 但由kubelet发起的驱逐只停止Pod的所有容器进程，并不会直接删除Pod。
 - Pod的status.phase会被标记为Failed
 - status.reason会被设置为Evicted
 - status.message则会记录被驱逐的原因

资源可用额监控

- kubelet依赖内嵌的开源软件cAdvisor，周期性检查节点资源使用情况
- CPU是可压缩资源，根据不同进程分配时间配额和权重，CPU可被多个进程竞相使用
- 驱逐策略是基于磁盘和内存资源用量进行的，因为两者属于不可压缩的资源，当此类资源使用耗尽时将无法再申请

检查类型	说明
memory.available	节点当前的可用内存
nodefs.available	节点根分区的可使用磁盘大小
nodefs.inodesFree	节点根分区的可使用inode
imagefs.inodesFree	节点运行时分区的可使用inode
imagefs.available	节点运行时分区的可使用磁盘大小 节点如果没有运行时分区，就不会有相应的资源监控

驱逐策略

kubelet获得节点的可用额信息后，会结合节点的容量信息来判断当前节点运行的Pod是否满足驱逐条件。

驱逐条件可以是绝对值或百分比，当监控资源的可使用额少于设定的数值或百分比时，kubelet就会发起驱逐操作。

kubelet参数evictionMinimumReclaim可以设置每次回收的资源的最小值，以防止小资源的多次回收。

kubelet参数	分类	驱逐方式
evictionSoft	软驱逐	当检测到当前资源达到软驱逐的阈值时，并不会立即启动驱逐操作，而是要等待一个宽限期。 这个宽限期选取EvictionSoftGracePeriod和Pod指定的TerminationGracePeriodSeconds中较小的值
evictionHard	硬驱逐	没有宽限期，一旦检测到满足硬驱逐的条件，就直接中止容器来释放紧张资源

基于内存压力的驱逐

memory.available表示当前系统的可用内存情况。

kubelet默认设置了memory.available<100Mi的硬驱逐条件

当kubelet检测到当前节点可用内存资源紧张并满足驱逐条件时，会将节点的MemoryPressure状态设置为True，调度器会阻止BestEffort Pod调度到内存承压的节点。

kubelet启动对内存不足的驱逐操作时，会依照如下的顺序选取目标Pod：

- （1）判断Pod所有容器的内存使用量总和是否超出了请求的内存量，超出请求资源的Pod会成为备选目标。
- （2）查询Pod的调度优先级，低优先级的Pod被优先驱逐。
- （3）计算Pod所有容器的内存使用量和Pod请求的内存量的差值，差值越小，越不容易被驱逐。

基于磁盘压力的驱逐

以下任何一项满足驱逐条件时，它会将节点的DiskPressure状态设置为True，调度器不会再调度任何Pod到该节点上

- nodefs.available
- nodefs.inodesFree
- imagefs.available
- imagefs.inodesFree

驱逐行为

- 有容器运行时分区
 - nodefs达到驱逐阈值，那么kubelet删除已经退出的容器
 - Imagefs达到驱逐阈值，那么kubelet删除所有未使用的镜像。
- 无容器运行时分区
 - kubelet同时删除未运行的容器和未使用的镜像。

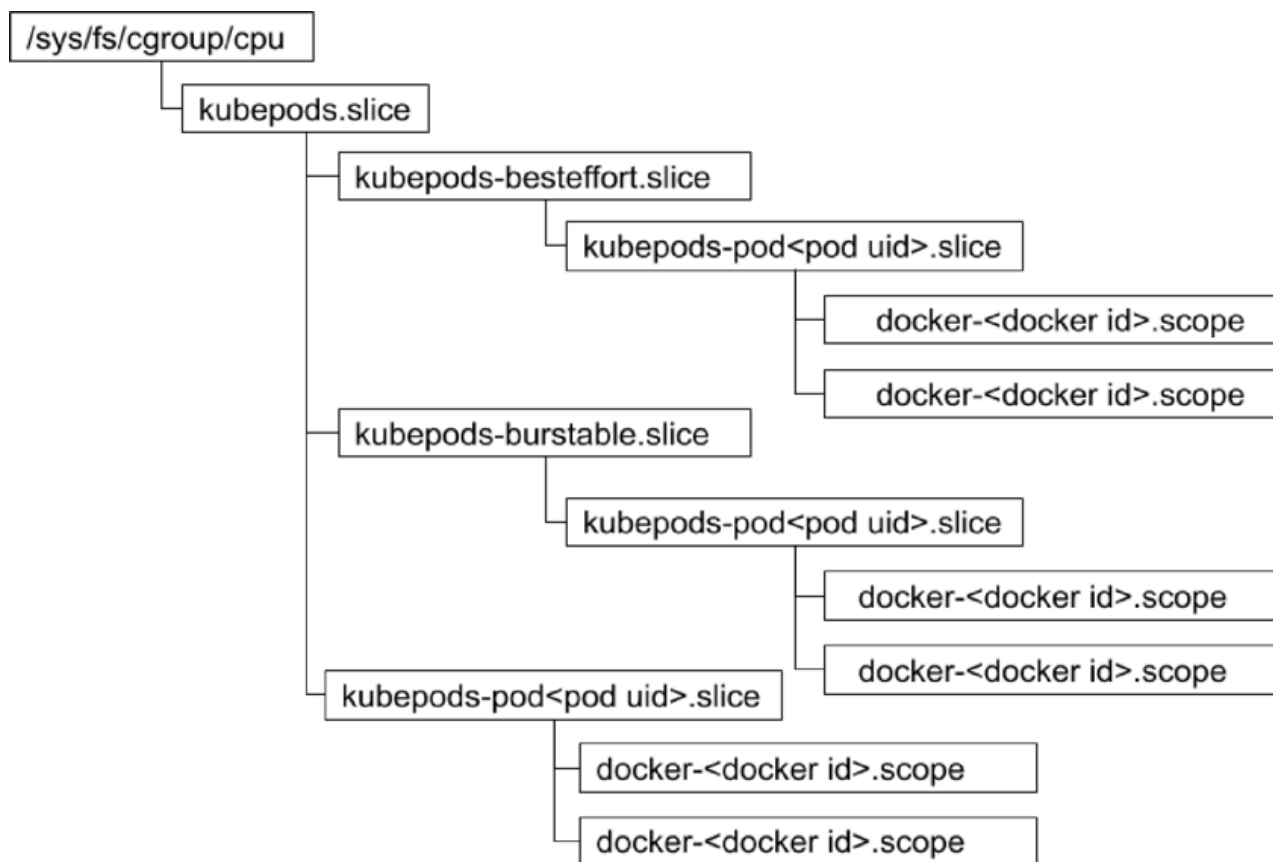
基于磁盘压力的驱逐

回收已经退出的容器和未使用的镜像后，如果节点依然满足驱逐条件，kubelet就会开始驱逐正在运行的Pod，进一步释放磁盘空间。

- 判断Pod的磁盘使用量是否超过请求的大小，超出请求资源的Pod会成为备选目标。
- 查询Pod的调度优先级，低优先级的Pod优先驱逐。
- 根据磁盘使用超过请求的数量进行排序，差值越小，越不容易被驱逐。

容器和资源配置

针对不同QoS Class的Pod, Kuberetes按如下Hierarchy组织cgroup中的CPU 子系统

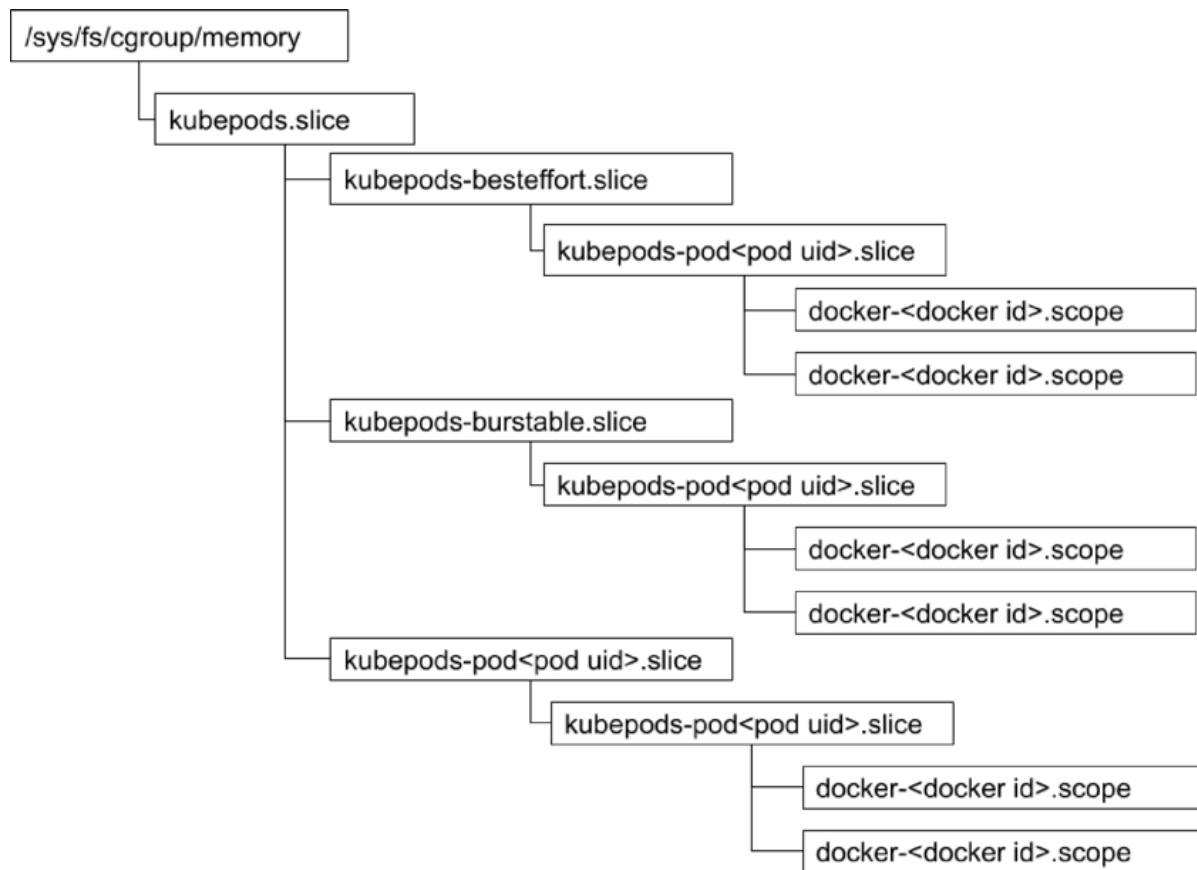


CPU CGroup配置

CGroup类型	参数	QoS类型	值
容器的CGroup	cpu.shares	BestEffort	2
		Burstable	requests.cpu×1024
		Guaranteed	requests.cpu×1024
	cpu.cfs_quota_us	BestEffort	-1
		Burstable	limits.cpu×100
		Guaranteed	limits.cpu×100
Pod的CGroup	cpu.shares	BestEffort	2
		Burstable	Pod所有容器（requests.cpu×1024）之和
		Guaranteed	Pod所有容器（requests.cpu×1024）之和
	cpu.cfs_quota_us	BestEffort	-1
		Burstable	Pod所有容器（limits.cpu×100）之和
		Guaranteed	Pod所有容器（limits.cpu×100）之和

容器和资源配置

针对不同QoS Class的Pod, Kubneretes按如下Hierarchy组织cgroup中的Memory 子系统



内存CGroup配置

CGroup类型	参数	QoS类型	值
容器的CGroup	memory.limit_in_bytes	BestEffort	9223372036854771712
		Burstable	limits.memory
		Guaranteed	limits.memory
Pod的CGroup	memory.limit_in_bytes	BestEffort	9223372036854771712
		Burstable	所有Pod容器（limits.memory）之和
		Guaranteed	所有Pod容器（limits.memory）之和

OOM Killer行为

- 系统的OOM Killer可能会采取OOM的方式来中止某些容器的进程，进行必要的内存回收操作
- 而系统根据进程的oom_score来进行优先级排序，选择待终止的进程，且进程的oom_score越高，越容易被终止。
- 进程的oom_score是根据当前进程使用的内存占节点总内存的比例值乘以10，再加上oom_score_adj综合得到的
- 而容器进程的oom_score_adj正是kubelet根据memory.request进行设置的

Pod QoS类型	oom_score_adj
Guaranteed	-998
BestEffort	1000
Burstable	$\min(\max(2, 1000 - (1000 \times \text{memoryRequestBytes}) / \text{machineMemoryCapacityBytes}), 999)$

9.1 测试对cpu的校验和准入行为

定义一个Pod，并将该Pod中的nodeName属性直接写成集群中的节点名

将Pod的CPU的资源设置为超出计算节点的CPU的值

创建该Pod

观察行为并思考

日志管理

节点上需要通过运行logrotate的定时任务对系统服务日志进行rotate清理，以防止系统服务日志占用大量的磁盘空间。

- logrotate的执行周期不能过长，以防日志短时间内大量增长。
- 同时配置日志的rotate条件，在日志不占用太多空间的情况下，保证有足够的日志可供查看。
- Docker
 - 除了基于系统logrotate管理日志，还可以依赖Docker自带的日志管理功能来设置容器日志的数量和每个日志文件的大小。
 - Docker写入数据之前会对日志大小进行检查和rotate操作，确保日志文件不会超过配置的数量和大小。
- Containerd
 - 日志的管理是通过kubelet定期（默认为10s）执行du命令，来检查容器日志的数量和文件的大小的。
 - 每个容器日志的大小和可以保留的文件个数，可以通过kubelet的配置参数container-log-max-size 和container-log-max-files进行调整。

Docker卷管理

- 在构建容器镜像时，可以在Dockerfile中通过VOLUME指令声明一个存储卷，目前Kubernetes尚未将其纳入管控范围，不建议使用。
- 如果容器进程在可写层或emptyDir卷进行大量读写操作，就会导致磁盘I/O过高，从而影响其他容器进程甚至系统进程。
- Docker和Containerd运行时都基于CGroup v1。对于块设备，只支持对Direct I/O限速，而对于Buffer I/O还不具备有效的支持。因此，针对设备限速的问题，目前还没有完美的解决方案，对于有特殊I/O需求的容器，建议使用独立的磁盘空间。

网络资源

由网络插件通过Linux Traffic Control为Pod限制带宽
可利用CNI社区提供的bandwidth插件

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubernetes.io/ingress-bandwidth: 10MB
    kubernetes.io/egress-bandwidth: 10MB
...
```


进程数

kubelet默认不限制Pod可以创建的子进程数量，但可以通过启动参数podPidsLimit开启限制，还可以由reserved参数为系统进程预留进程数。

- kubelet通过系统调用周期性地获取当前系统的PID的使用量，并读取 /proc/sys/kernel/pid_max，获取系统支持的PID上限。
- 如果当前的可用进程数少于设定阈值，那么kubelet会将节点对象的PIDPressure标记为True
- kube-scheduler在进行调度时，会从备选节点中对处于NodeUnderPIDPressure状态的节点进行过滤。

4. 节点异常检测

Kubernetes集群可能存在的问题

基础架构守护程序问题：NTP服务关闭；

硬件问题：CPU，内存或磁盘损坏；

内核问题：内核死锁，文件系统损坏；

容器运行时问题：运行时守护程序无响应

...

当kubernetes中节点发生上述问题，在整个集群中，k8s服务组件并不会感知以上问题，就会导致pod仍会调度至问题节点。

node-problem-detector

为了解决这个问题，社区引入了守护进程node-problem-detector，从各个守护进程收集节点问题，并使它们对上游层可见。

Kubernetes节点诊断的工具，可以将节点的异常，例如

- Runtime 无响应
- Linux Kernel 无响应
- 网络异常
- 文件描述符异常
- 硬件问题如CPU，内存或者磁盘故障

故障分类

Problem Daemon Types	NodeCondition	Description	Configs
系统日志监控	KernelDeadlock ReadonlyFilesystem FrequentKubeletRestart FrequentDockerRestart FrequentContainerdRestart	通过监控系统日志来汇报问题并输出系统指标数据	filelog, kmsg, kernel a brt systemd
CustomPluginMonitor	按需定义	自定义插件监控允许用户自定义监控脚本，并运行这些脚本来进行监控	比如ntp服务监控
HealthChecker	KubeletUnhealthy ContainerRuntimeUnhealthy	针对kubelet和运行时的健康检查	kubelet docker

问题汇报手段

node-problem-detector通过设置NodeCondition或者创建Event对象来汇报问题

- NodeCondition: 针对永久性故障, 会通过设置NodeCondition来改变节点状态
- Event: 临时故障通过Event来提醒相关对象, 比如通知当前节点运行的所有Pod

上手实践

代码

<https://github.com/kubernetes/node-problem-detector>

安装

```
helm repo add deliveryhero https://charts.deliveryhero.io/
```

```
helm install deliveryhero/node-problem-detector
```

使用插件 pod 启用NPD

如果你使用的是自定义集群引导解决方案，不需要覆盖默认配置，可以利用插件 Pod 进一步自动化部署。

创建 node-strick-detector.yaml，并在控制平面节点上保存配置到插件 Pod 的目录 /etc/kubernetes/addons/node-problem-detector。

NPD的异常处理行为

- NPD只负责获取异常事件，并修改node condition，不会对节点状态和调度产生影响
lastHeartbeatTime: "2021-11-06T15:44:46Z"
lastTransitionTime: "2021-11-06T15:29:43Z"
message: 'kernel: INFO: task docker:20744 blocked for more than 120 seconds.'
reason: DockerHung
status: "True"
type: KernelDeadlock
- 需要自定义控制器，监听NPD汇报的condition，taint node，阻止pod调度到故障节点
- 问题修复后，重启NPD Pod来清理错误事件

5. 常用节点问题排查手段

ssh到内网节点

- 创建一个支持ssh的pod
- 并通过负载均衡器转发ssh请求

查看日志

针对使用systemd拉起的服务

```
journalctl -afu kubelet -S "2019-08-26 15:00:00"  
-u unit, 对应的systemd拉起的组件, 如kubelet  
-f follow, 跟踪最新日志  
-a show all, 现实所有日志列  
-S since, 从某一时间开始 -S "2019-08-26 15:00:00"
```

对于标准的容器日志

```
kubectrl logs -f -c <containername> <podname>  
kubectrl logs -f --all-containers <podname>  
kubectrl logs -f -c <podname> --previous
```

如果容器日志被shell转储到文件, 则需通过exec

```
kubectrl exec -it xxx --tail -f /path/to/log
```

6. 基于extended resource扩展节点资源

扩展资源

扩展资源是 `kubernetes.io` 域名之外的标准资源名称。它们使得集群管理员能够颁布非 Kubernetes 内置资源，而用户可以使用他们。

自定义扩展资源无法使用 `kubernetes.io` 作为资源域名

管理扩展资源

- 节点级扩展资源
 - 节点级扩展资源绑定到节点
- 设备插件管理的资源
 - 发布在各节点上由设备插件所管理的资源，如GPU，智能网卡等

为节点配置资源

- 集群操作员可以向 API 服务器提交 PATCH HTTP 请求， 以在集群中节点的状态.capacity 中为其配置可用数量。
 - 完成此操作后， 节点的状态.capacity 字段中将包含新资源。
 - kubelet 会异步地对 status.allocatable 字段执行自动更新操作， 使之包含新资源。
 - 调度器在评估 Pod 是否适合在某节点上执行时会使用节点的状态.allocatable 值， 在更新节点容量使之包含新资源之后和请求该资源的第一个 Pod 被调度到该节点之间， 可能会有短暂的延迟。
-
- ```
curl --key admin.key --cert admin.crt --header "Content-Type: application/json-patch+json" \
 --request PATCH -k \
 --data '[{"op": "add", "path": "/status/capacity/cncamp.com~1reclaimed-cpu",
 "value": "2"}]' \
 https://192.168.34.2:6443/api/v1/nodes/cadmin/status
```



# 使用扩展资源

```
spec:
 containers:
 - name: nginx
 image: nginx
 resources:
 limits:
 cncamp.com/reclaimed-cpu: 3
 requests:
 cncamp.com/reclaimed-cpu: 3
```

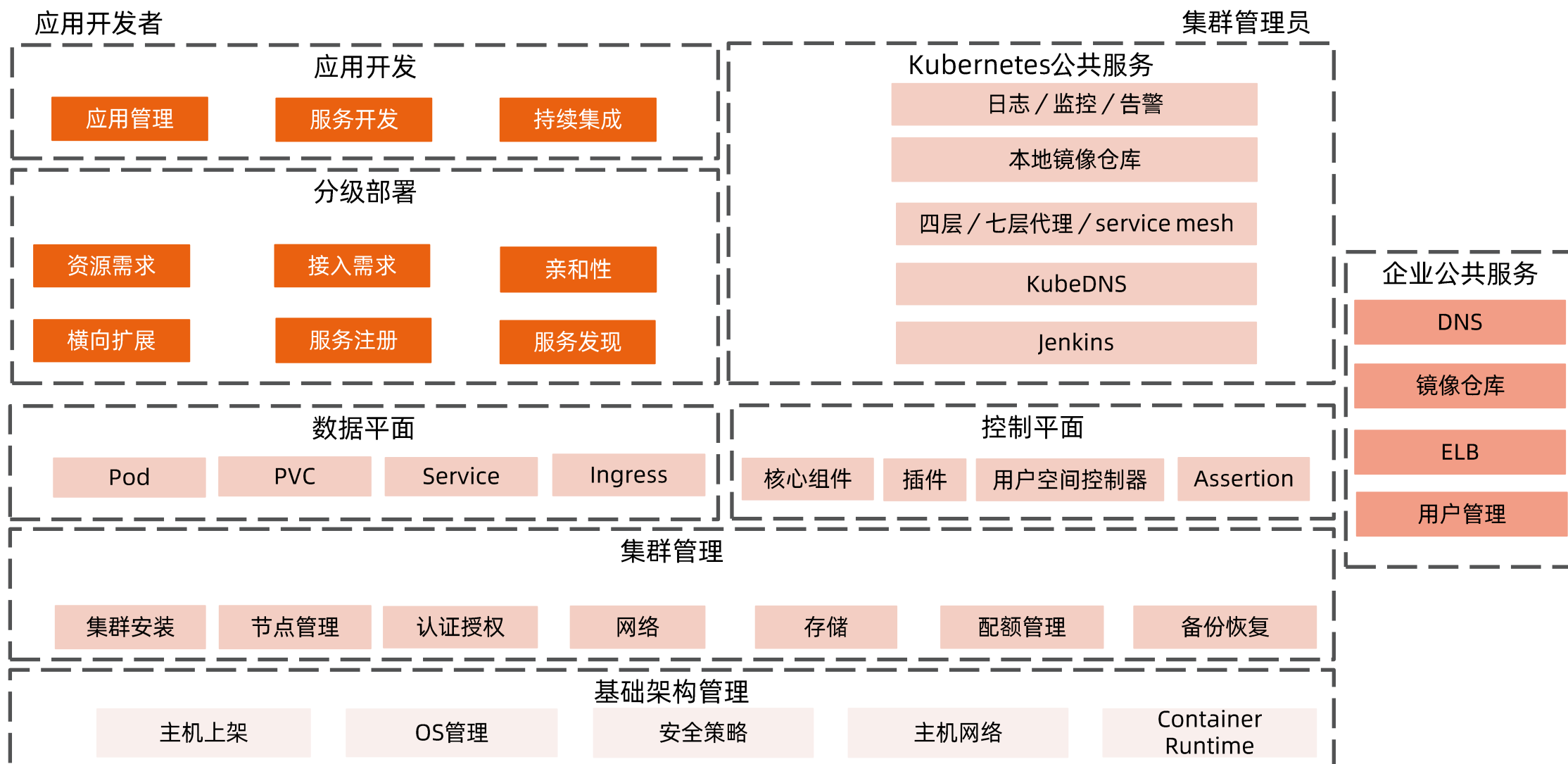
# 集群层面的扩展资源

- 可选择由默认调度器管理资源，默认调度器像管理其他资源一样管理扩展资源
  - Request与Limit必须一致，因为Kubernetes无法确保扩展资源的超售
- 更常见的场景是，由调度器扩展程序（Scheduler Extenders）管理，这些程序处理资源消耗和资源配额
  - 修改调度器策略配置ignoredByScheduler字段可配置调度器不要检查自定义资源

```
{
 "kind": "Policy",
 "apiVersion": "v1",
 "extenders": [
 {
 "urlPrefix": "<extender-endpoint>",
 "bindVerb": "bind",
 "managedResources": [
 {
 "name": "example.com/foo",
 "ignoredByScheduler": true
 }
]
 }
]
}
```

## 7. 构建和管理高可用集群

# Kubernetes高可用层级



# 高可用的数据中心

- 多地部署
- 每个数据中心需要划分成具有独立供电、制冷、网络设备的高可用区
- 每个高可用区管理独立的硬件资产，包括机架、计算节点、存储、负载均衡器、防火墙等硬件设备

# Node的生命周期管理

运营Kubernetes集群，不仅仅是集群搭建那么简单，运营需要对集群中所有节点的完整申明周期负责。

- 集群搭建
- 集群扩容 / 缩容
- 集群销毁（很少）
- 无论是集群搭建还是扩容，核心是Node的生命周期管理
  - Onboard
    - 物理资产上架
    - 操作系统安装
    - 网络配置
    - Kubernetes组件安装
    - 创建Node 对象
  - 故障处理
    - 临时故障？重启大法好
    - 永久故障？机器下架
  - Offboard
    - 删除Node对象
    - 物理资产下架，送修 / 报废

# 主机管理

- 选定哪个版本的系统内核、哪个发行版、安装哪些工具集、主机网络如何规划等。
- 日常的主机镜像升级更新也可能是造成服务不可用的因素之一。
  - 主机镜像更新可以通过A/B系统OTA（Over The Air）升级方式进行。
    - 分别使用A、B两个存储空间，共享一份用户数据。在升级过程中，OTA更新即往其中一个存储空间写入升级包，同时保证了另一个系统可以正常运行，而不会打断用户。如果OTA失败，那么设备会启动到OTA之前的磁盘分区，并且仍然可以使用。

# 生产化集群管理

- 如何设定单个集群规模
  - 社区声明单一集群可支持5000节点，在如此规模的集群中，大规模部署应用是有诸多挑战的。应该更多还是更少？如何权衡？
- 如何根据地域划分集群
  - 不同地域的计算节点划分到同一集群
  - 将同一地域的节点划分到同一集群
- 如何规划集群的网络
  - 企业办公环境、测试环境、预生产环境和生产环境应如何进行网络分离
  - 不同租户之间应如何进行网络隔离
- 如何自动化搭建集群
  - 如何自动化搭建和升级集群，包括自动化部署控制平面和数据平面的核心组件
  - 如何与企业的公共服务集成



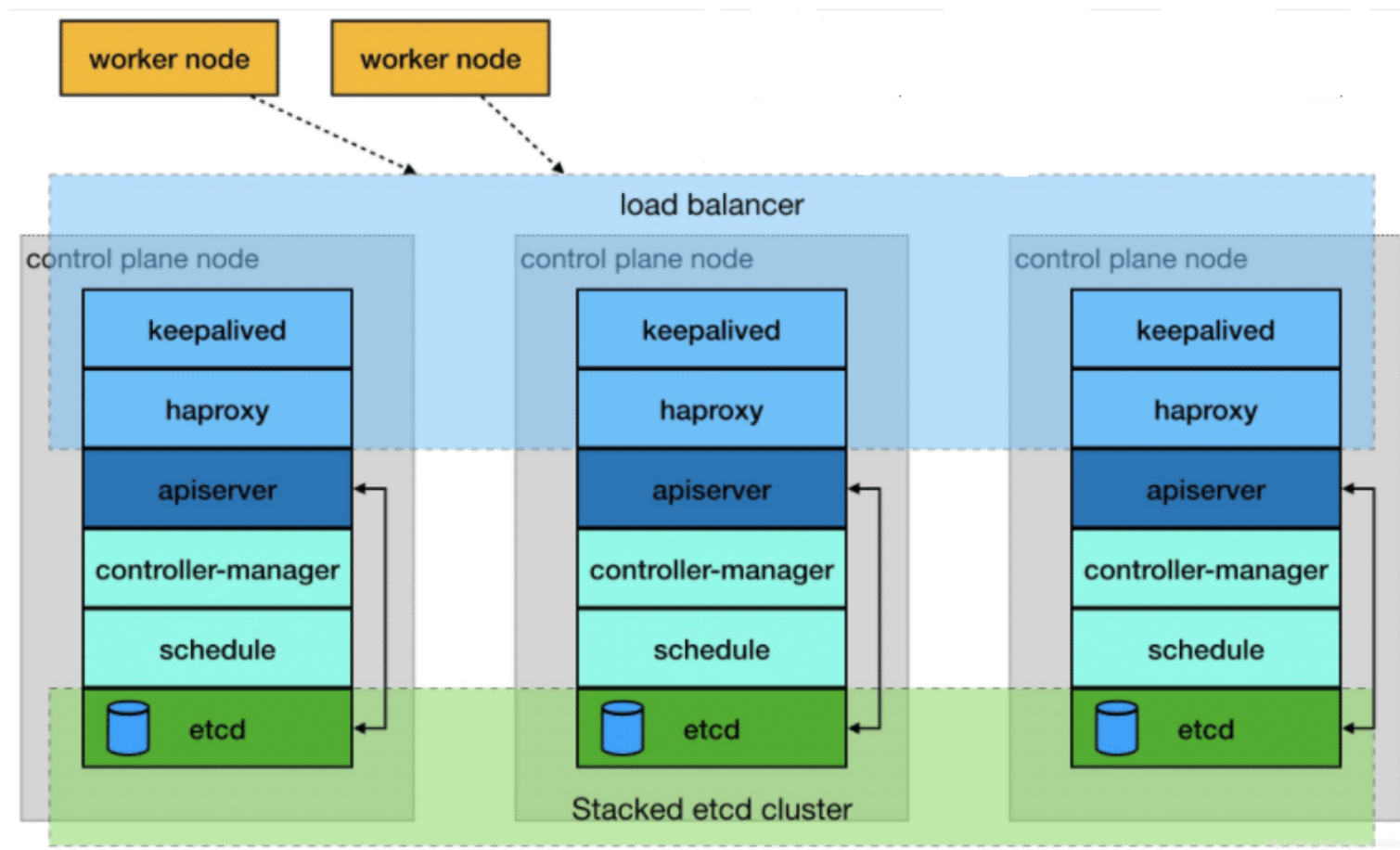
# 企业公共服务

- 需要与企业认证平台集成，这样企业用户就能通过统一认证平台接入Kubernetes集群，而无须重新设计和管理一套用户系统。
- 集成企业的域名服务、负载均衡服务，提供集群服务对企业外发布的访问入口
- 在与企业的公共服务集成时，需要考虑它们的服务是否可靠
- 对于不能异步调用的请求，采用同步调用需要设置合理的超时时间
- 过长的超时时间，会延迟结果等待时间，导致整体的链路调用时间延长，从而降低整体的TPS
- 有些失败是短暂的、偶然的（比如网络抖动），进行重试即可。而有些失败是必然的，重试反而会造成调用请求量放大，加重对调用系统的负担

# 控制平面的高可用保证

- 针对大规模的集群，应该为控制平面组件划分单独节点，减少业务容器对控制平面容器或守护进程的干扰和资源抢占
- 控制平面所在的节点，应确保在不同机架上，以防止因为某些机架的交换机或电源出问题，造成所有的控制面节点都无法工作
- 保证控制平面的每个组件有足够的CPU、内存和磁盘资源，过于严苛的资源限制会导致系统效率低下，降低集群可用性
- 应尽可能地减少或消除外部依赖。在Kubneretes初期版本中存在较多Cloud Provider API的调用，导致在运营过程中，当Cloud Provider API出现故障时，会使得Kubernetes集群也无法正常工作。
- 应尽可能地将控制平面和数据平面解耦，确保控制平面组件出现故障时，将业务影响降到最低。
- Kubernetes还有一些核心插件，是以普通的Pod形式加载运行的，可能会被调度到任意工作节点，与普通应用竞争资源。这些插件是否正常运行也决定了集群的可用性。

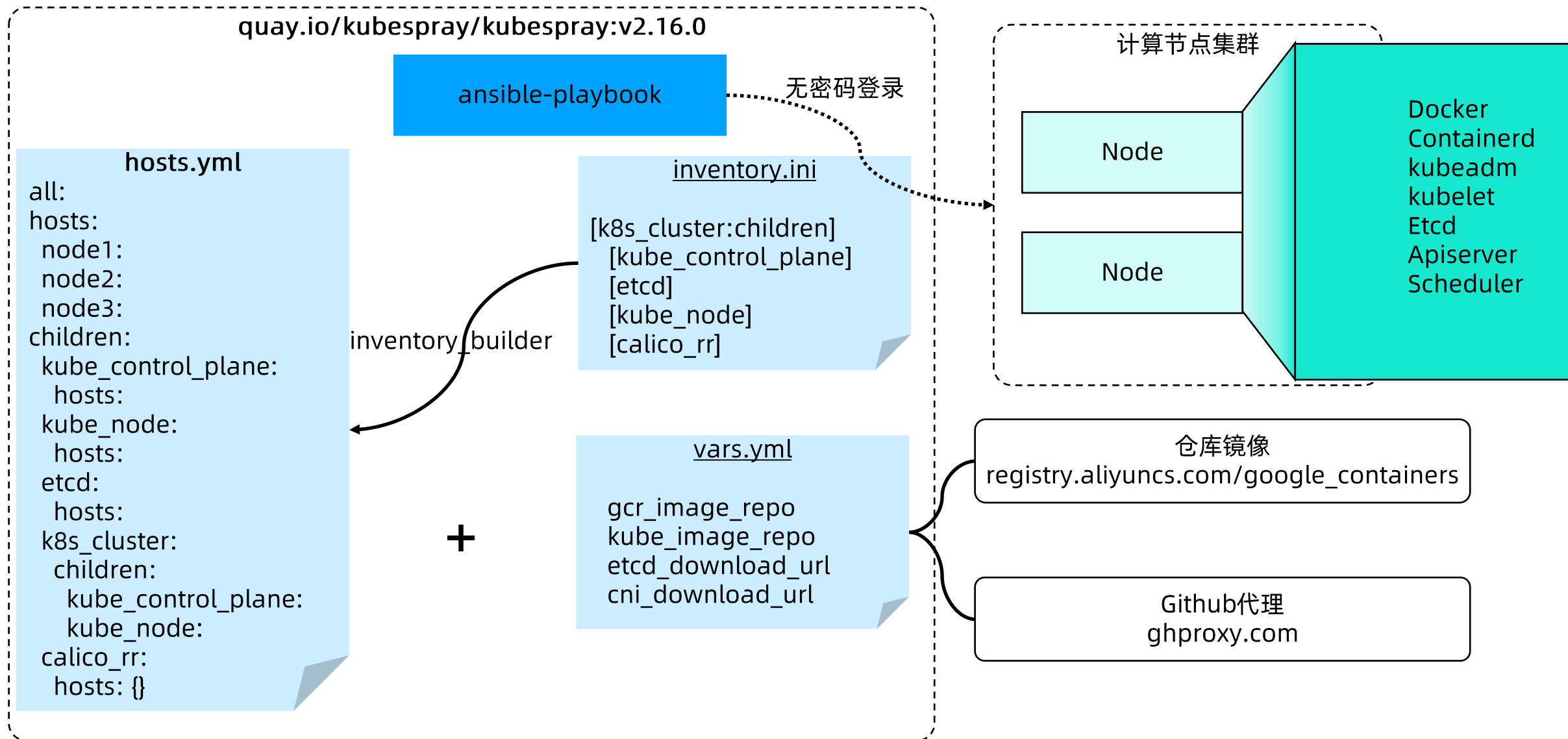
# 高可用集群



# 集群安装方法比较

| 安装工具      | 方法                                                                    | 优势                                               | 缺点                                                  |
|-----------|-----------------------------------------------------------------------|--------------------------------------------------|-----------------------------------------------------|
| 二进制       | 下载二进制文件，并通过设置systemd来管理                                               | 灵活度强                                             | 复杂，需要关心每一个组件的配置<br>对系统服务的依赖性过多                      |
| Kubeadm   | Kubeadm是一个搭建集群的命令行工具<br>管理节点通过kubeadm init初始化<br>计算节点通过kubeadm join加入 | 相比二进制，控制面板组件的安装和配置被封装起来了<br>管理集群的生命周期，比如升级，证书管理等 | 操作系统层面的配置无自动化<br>运行时安装配置等复杂步骤依然是必须的<br>CNI插件等需要手工安装 |
| Kubespray | 通过Ansible-playbook完成集群搭建                                              | 自动完成操作系统层面的配置<br>利用了kubeadm作为集群管理工具              | 缺少基于声明式API的支持                                       |
| KOPS      | 基于声明式API的集群管理工具                                                       | 基于社区标准的Cluster API进行集群管理<br>节点的操作系统安装等全自动化       | 与云环境深度集成<br>灵活性差                                    |

# 用Kubespray搭建高可用集群搭建



## 练习9.2：用Kubespray安装集群

按照如下的教程，用Kubespray安装集群

示例中展示了2节点的安装方法，可按照你有的环境灵活决定装多少个节点的集群

<https://pouncing-waterfall-7c4.notion.site/Install-HA-cluster-using-kubespray-e398756229a649b8adc7338788df4672>

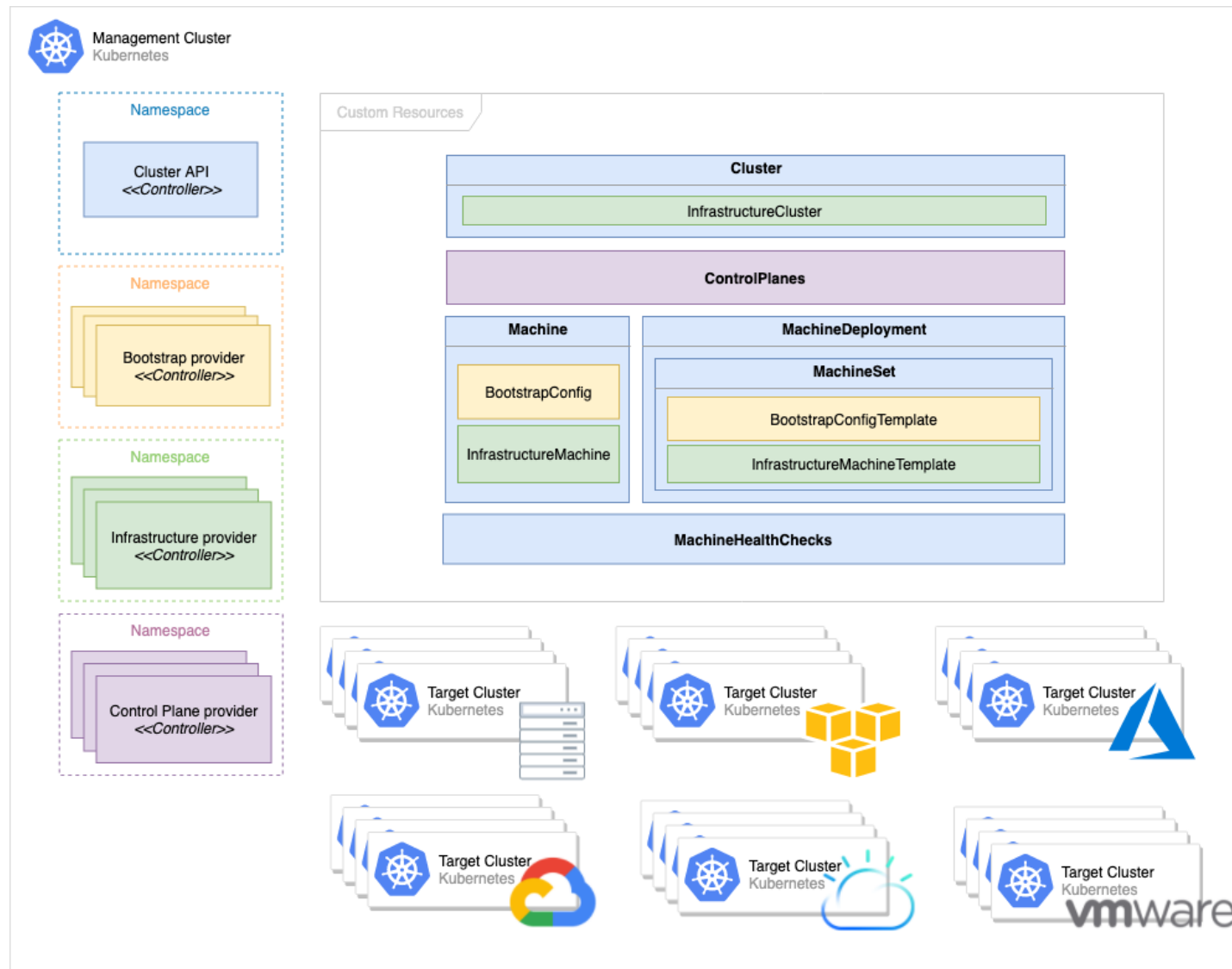
# 基于声明式API管理集群

集群管理不仅仅包括集群搭建，还有更多功能需要支持

- 集群扩缩容
- 节点健康检查和自动修复
- Kubernetes升级
- 操作系统升级

云原生场景中集群应该按照我们的期望的状态运行，这意味着我们应该将集群管理建立在声明式API的基础之上

# Kubernetes Cluster API





# 参与角色

- 管理集群
  - 管理workload 集群的集群，用来存放Cluster API对象的地方
- Workload集群
  - 真正开放给用户用来运行应用的集群，由管理集群管理
- Infrastructure provider
  - 提供不同云的基础架构管理，包括计算节点，网络等。目前流行的公有云多与Cluster API集成了。
- Bootstrap provider
  - 证书生成
  - 控制面组件安装和初始化，监控节点的创建
  - 将主节点和计算节点加入集群
- Control plane
  - Kubernetes控制平面组件

# 涉及模型

- Machine
  - 计算节点，用来描述可以运行Kubernetes组件的机器对象（注意与Kubernetes Node）的差异
  - 一个新Machine被创建以后，对应的控制器会创建一个计算节点，安装好操作系统并更新Machine的状态
  - 当一个Machine被删除后，对应的控制器会删除掉该节点并回收计算资源。
  - 当Machine属性被更新以后（比如Kubernetes版本更新），对应的控制器会删除旧节点并创建新节点
- Machine Immutability (In-place Upgrade vs. Replace)
  - 不可变基础架构
- MachineDeployment
  - 提供针对Machine和MachineSet的声明式更新，类似于Kubernetes Deployment
- MachineSet
  - 维护一个稳定的机器集合，类似Kubernetes ReplicaSet
- MachineHealthCheck
  - 定义节点应该被标记为不可用的条件

# 用cluster api管理集群

### create host cluster

```
./create_cluster.sh
```

### generate cluster specs

```
cd cluster-api
```

```
./generate_workload_cluster.sh
```

### replace image repository

```
vi capi-quickstart.yaml
```

```
replace KubeadmConfigTemplate
```

### apply cluster spec

```
kubectl apply -f capi-quickstart.yaml
```

```
apiVersion: bootstrap.cluster.x-k8s.io/v1alpha4
kind: KubeadmConfigTemplate
metadata:
 name: capi-quickstart-md-0
 namespace: default
spec:
 template:
 spec:
 joinConfiguration:
 nodeRegistration:
 kubeletExtraArgs:
 cgroup-driver: cgroupfs
 eviction-hard:
nodefs.available<0%,nodefs.inodesFree<0%,imagefs.available<0%
clusterConfiguration:
 imageRepository: registry.aliyuncs.com/google_containers
```

# KubeadmControlPlane

apiVersion: controlplane.cluster.x-k8s.io/v1alpha4

kind: KubeadmControlPlane

metadata:

name: capi-quickstart-control-plane

spec:

kubeadmConfigSpec:

clusterConfiguration:

apiServer:

certSANs:

- localhost

- 127.0.0.1

controllerManager:

extraArgs:

enable-hostpath-provisioner: "true"

dns: {}

etcd: {}

networking: {}

scheduler: {}

joinConfiguration:

discovery: {}

nodeRegistration:

criSocket: /var/run/containerd/containerd.sock

kubeletExtraArgs:

cgroup-driver: cgroupfs

eviction-hard: nodefs.available<0%,nodefs.inodesFree<0%,imagefs.available<0%

initConfiguration:

localAPIEndpoint: {}

nodeRegistration:

criSocket: /var/run/containerd/containerd.sock

kubeletExtraArgs:

cgroup-driver: cgroupfs

eviction-hard: nodefs.available<0%,nodefs.inodesFree<0%,imagefs.available<0%

machineTemplate:

infrastructureRef:

apiVersion: infrastructure.cluster.x-k8s.io/v1alpha4

kind: DockerMachineTemplate

name: capi-quickstart-control-plane

namespace: default

metadata: {}

replicas: 1

rolloutStrategy:

rollingUpdate:

maxSurge: 1

type: RollingUpdate

version: v1.22.0

# MachineDeployment

apiVersion: cluster.x-k8s.io/v1alpha4

kind: MachineDeployment

metadata:

name: capi-quickstart-md-0

spec:

strategy:

rollingUpdate:

maxSurge: 1

maxUnavailable: 0

type: RollingUpdate

template:

metadata:

labels:

cluster.x-k8s.io/cluster-name: capi-quickstart

cluster.x-k8s.io/deployment-name: capi-quickstart-md-0

spec:

bootstrap:

configRef:

apiVersion: bootstrap.cluster.x-k8s.io/v1alpha4

kind: KubeadmConfigTemplate

name: capi-quickstart-md-0

namespace: default

clusterName: capi-quickstart

infrastructureRef:

apiVersion: infrastructure.cluster.x-k8s.io/v1alpha4

kind: DockerMachineTemplate

name: capi-quickstart-md-0

namespace: default

version: v1.22.0

# MachineHealthCheck

apiVersion: cluster.x-k8s.io/v1alpha3

kind: MachineHealthCheck

metadata:

name: capi-quickstart-node-unhealthy-5m

spec:

clusterName: capi-quickstart

maxUnhealthy: 40%

自动修复节点数的上限，默认行为是替换该节点，云提供商可自定义

nodeStartupTimeout: 10m

selector:

matchLabels:

健康检查的目标节点

nodepool: nodepool-0

unhealthyConditions:

- type: Ready

status: Unknown

timeout: 300s

- type: Ready

认定节点不健康的条件

status: "False"

timeout: 300s

# 日常运营中的节点问题归类

## 可自动修复的问题

- 计算节点down
  - Ping不通
  - TCP probe 失败
  - 节点上的所有应用都不可达

## 不可自动修复的问题

- 文件系统坏
- 磁盘阵列故障
- 网盘挂载问题
- 其他硬件故障
- Kernel 出错, core dumps

## 其他问题

- 软件Bug
- 进程锁死, 或者memory/CPU 竞争问题
- Kubernetes组件出问题
  - Kubelet / Kube-proxy / Docker / Salt

# 故障监测和自动恢复

当创建Compute节点时，允许定义Liveness Probe

- 当livenessProbe失败时，ComputeNode的ProbePassed设置为false

在Prometheus中，已经有Node level的alert，抓取Prometheus中的alert

设定自动恢复规则

- 大多数情况下，重启大法好（人人都是Restart operator）。
- 如果重启不行就重装。（reprovision）
- 重装不行就重修。（breakfix）

```
spec:
 livenessProbe:
 tcpSocket:
 port: 22
 initialDelaySeconds: 15
 periodSeconds: 20
```

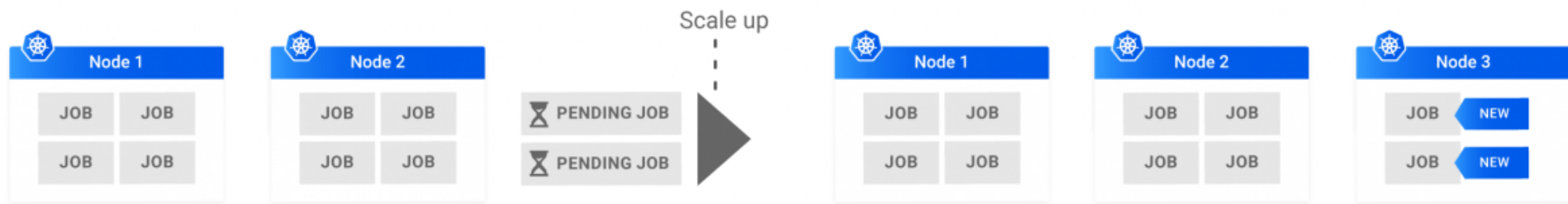
```
status:
 conditions:
 - lastProbeTime: 2018-02-08T22:37:12Z
 lastTransitionTime: 2018-01-23T19:37:30Z
 message: Active alerts Disk_Health_Status_Unhealthy
 found
 reason: ActiveAlertsFound
 status: "False"
 type: NoActiveAlert
```



## 8. Cluster Autoscaler

# 工作机制

- 扩容:
  - 由于资源不足, pod调度失败, 即有pod一直处于Pending状态
- 缩容
  - node的资源利用率较低时, 持续10分钟低于50%
  - 此node上存在的pod都能被重新调度到其他node上运行



# Cluster Autoscaler架构

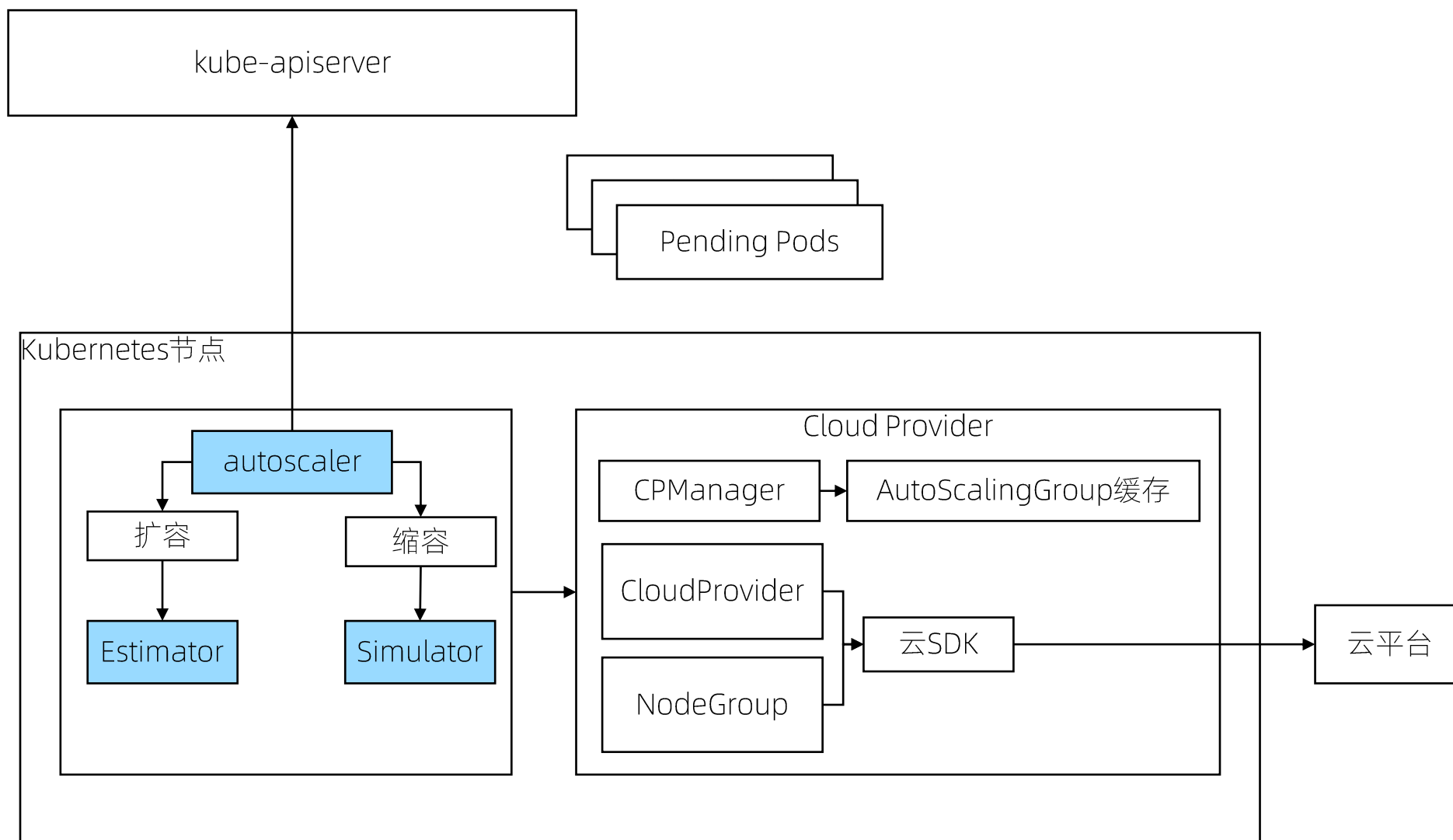
Autoscaler: 核心模块, 负责整体扩缩容功能

Estimator: 负责评估计算扩容节点

Simulator: 负责模拟调度, 计算缩容节点

Cloud-Provider: 与云交互进行节点的增删操作, 每个支持CA的主流厂商都实现自己的plugin实现动态缩放

# Cluster Autoscaler架构



# Cluster Autoscaler的扩展机制

为了自动创建和初始化 Node，Cluster Autoscaler 要求 Node 必须属于某个 Node Group，比如

- GCE/GKE 中的 Managed instance groups (MIG)
- AWS 中的 Autoscaling Groups
- Cluster API Node

当集群中有多个 Node Group 时，可以通过 `--expander=<option>` 选项配置选择 Node Group 的策略，支持如下四种方式

- random: 随机选择
- most-pods: 选择容量最大（可以创建最多 Pod）的 Node Group
- least-waste: 以最小浪费原则选择，即选择有最少可用资源的 Node Group
- price: 选择最便宜的 Node Group

# 附加资料

代码走读

<https://cncamp.notion.site/CA-204f3767ba8d4ed0a464d2ea6ac2abca>

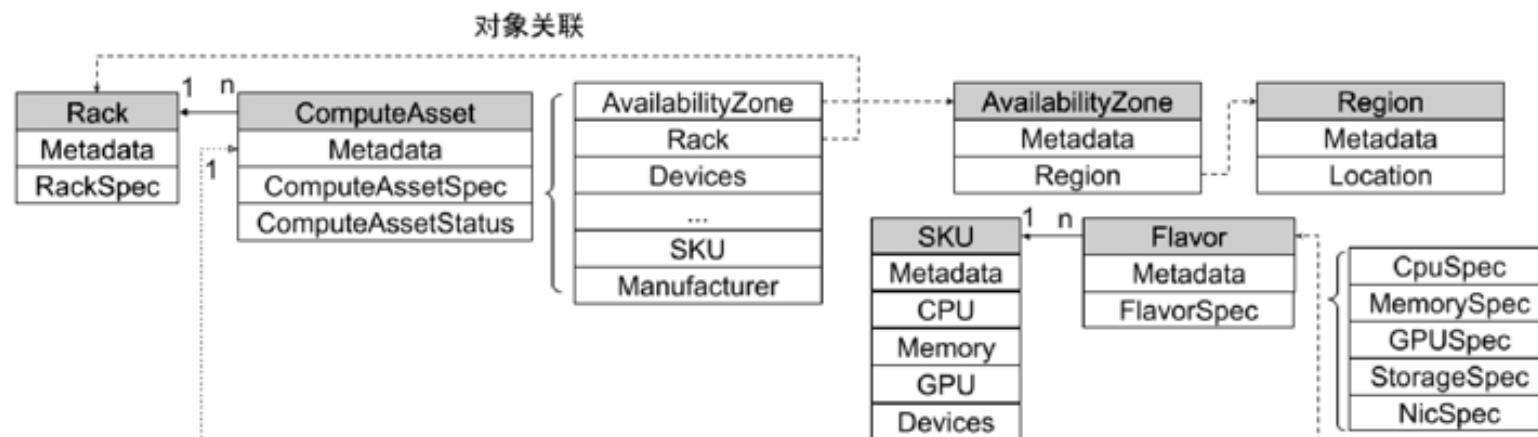
Cluster API与Cluster Autoscaler的整合

<https://cluster-api.sigs.k8s.io/tasks/cluster-autoscaler.html>

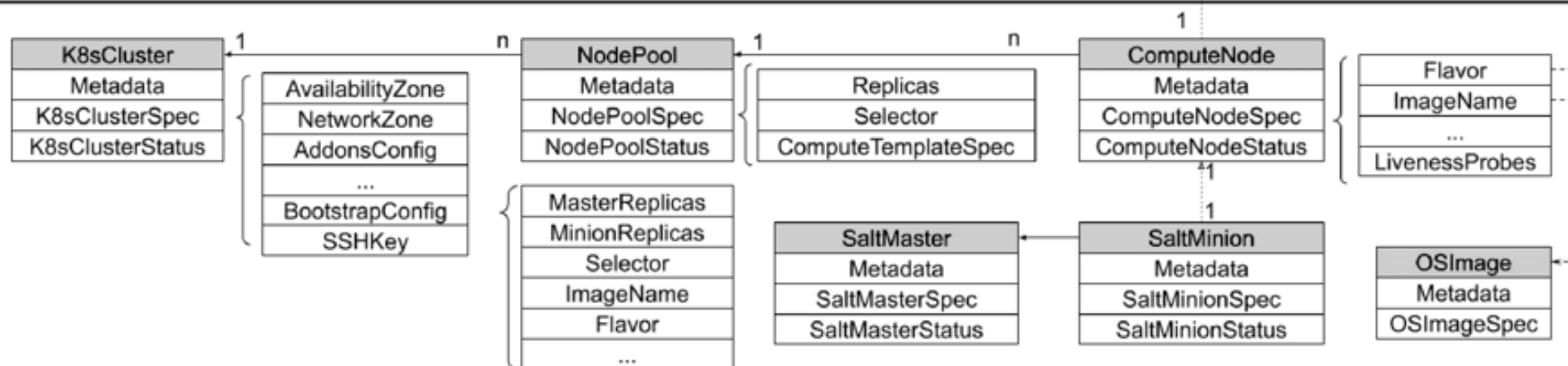
## 9. 集群管理实践案例分享

# 集群管理实践案例分享

设备模型

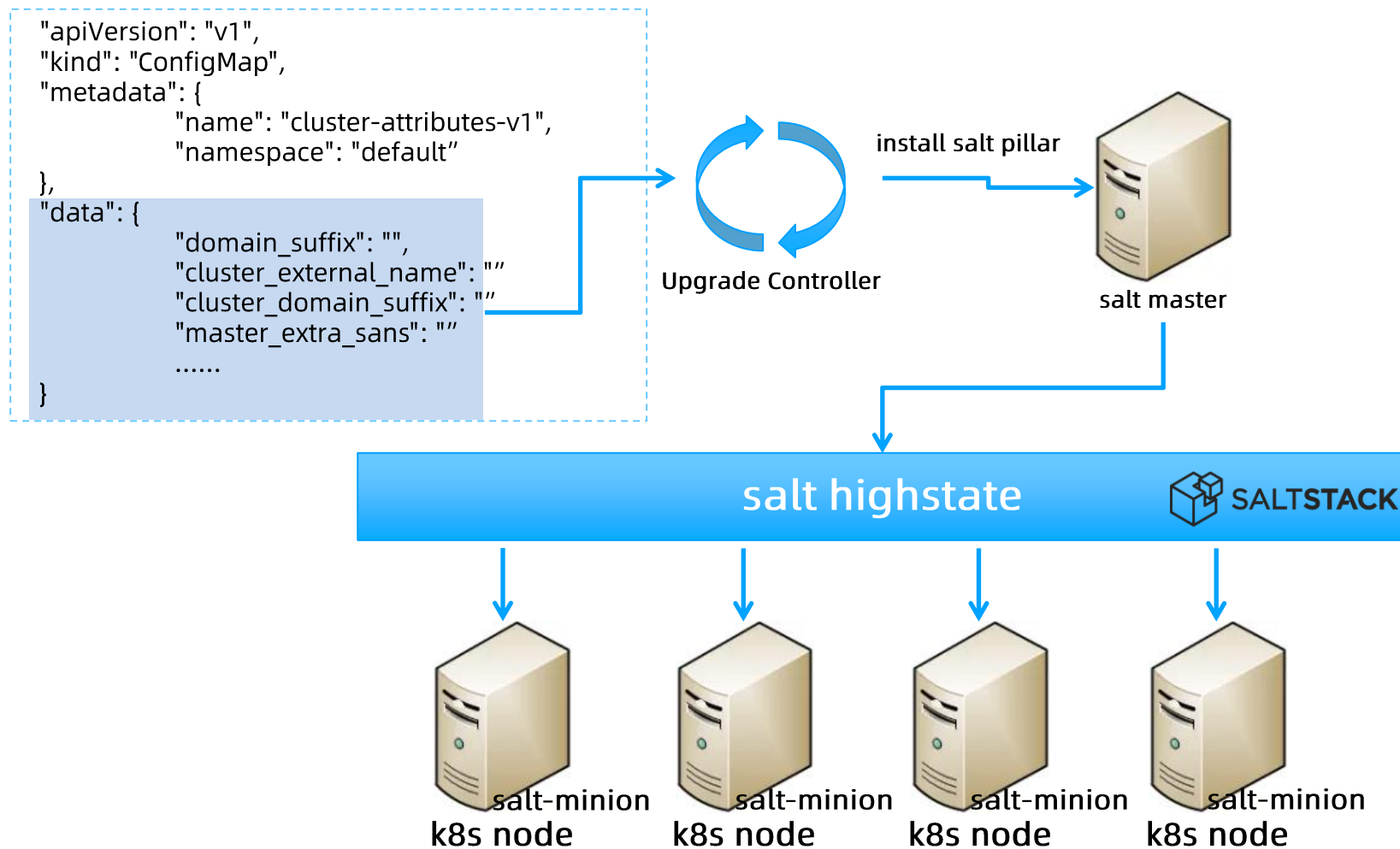


集群模型

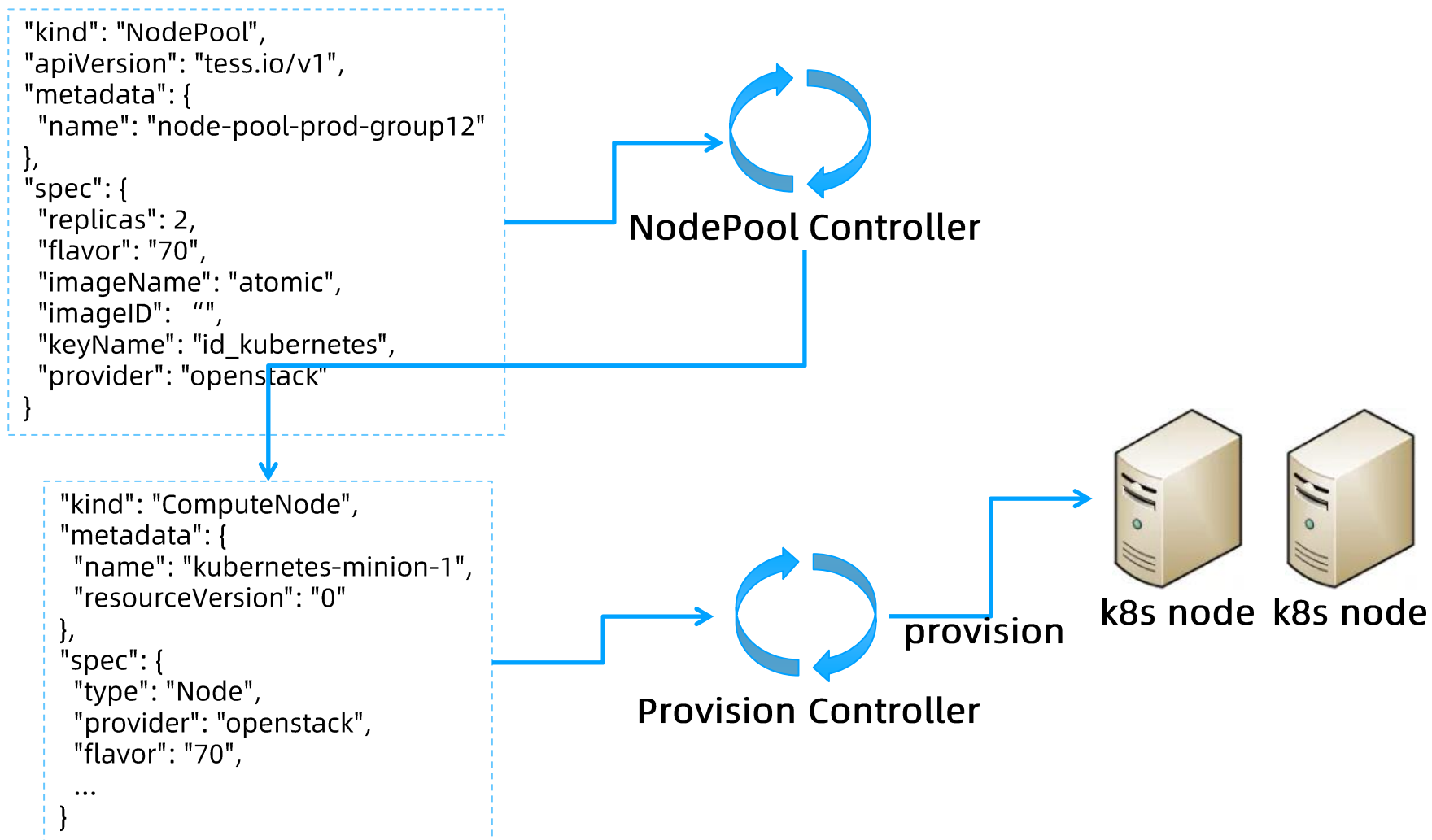




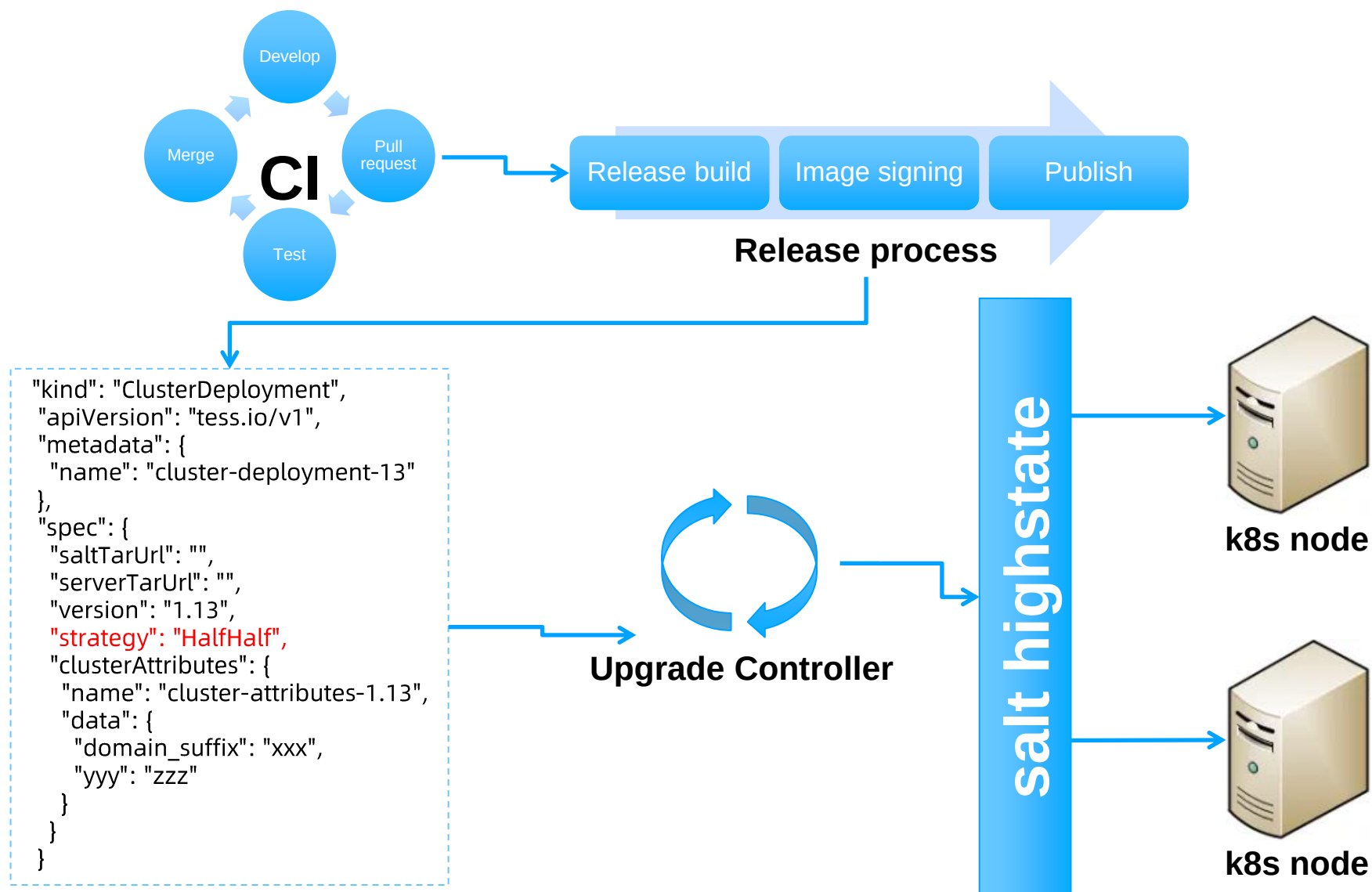
# 声明式集群配置



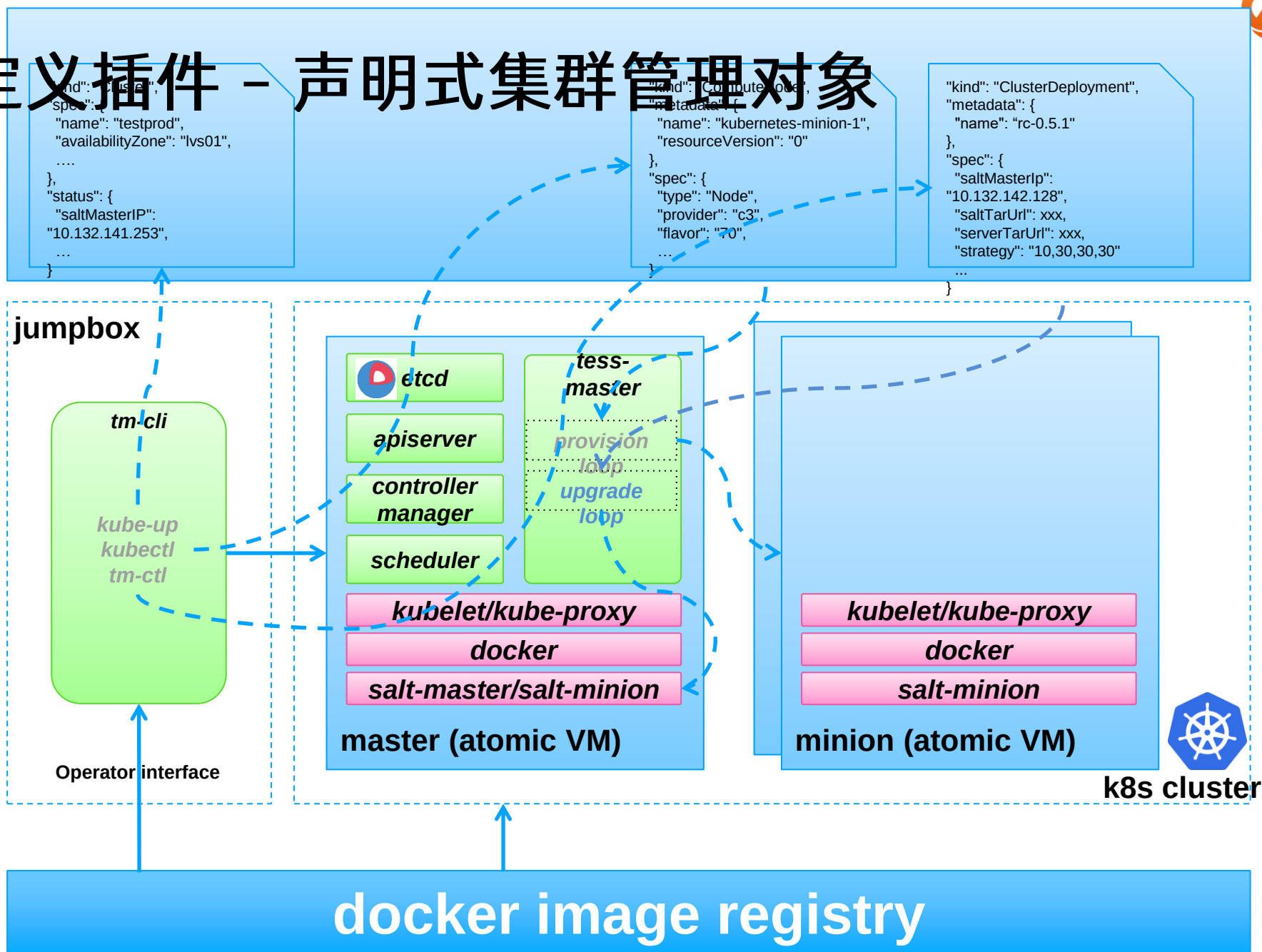
# 声明式扩容



# 声明式持续发布



# 自定义插件 - 声明式集群管理对象



## 10. 多租户集群管理

# 租户

- 租户是指一组拥有访问特定软件资源权限的用户集合，在多租户环境中，它还包括共享的应用、服务、数据和各项配置等
- 多租户集群必须将租户彼此隔离，以最大限度地减少租户与租户、租户与集群之间的影响
- 集群须在租户之间公平地分配集群资源。通过多租户共享集群资源，可以有效地降低集群管理成本，提高整体集群的资源利用率

# 认证-实现多租户的基础

- 租户管理首先需要识别访问的用户是谁，因此用户身份认证是多租户的基础
- 权限控制，如允许合法登录的用户访问、拒绝非法登录的用户访问或提供有限的匿名访问
- Kubernetes可管理两类用户
  - 用来标识和管理系统组件的ServiceAccount
  - 外部用户的认证，需要通过Kubernetes的认证扩展来对接企业、供应商的认证服务，为用户验证、操作授权、资源隔离等提供基础

# 隔离

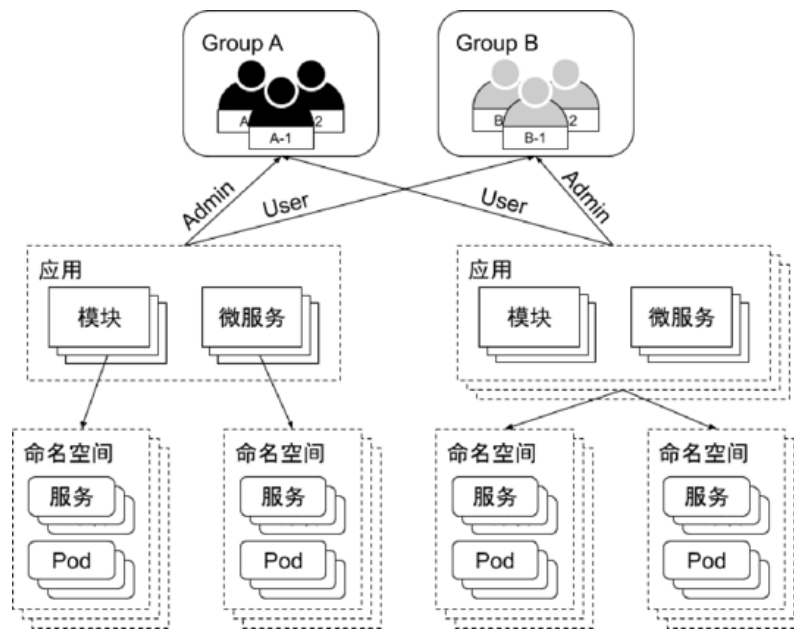
除认证、授权这些基础条件外，还要能够保证用户的工作负载彼此之间有尽可能安全的隔离，减少用户工作负载之间的影响。通常从权限、网络、数据三个方面对不同用户进行隔离

- 权限隔离
  - 普通用户的容器默认不具有privileged、sys\_admin、net\_admin等高级管理权限，以阻止对宿主机及其他用户的容器进行读取、写入等操作。
- 网络隔离
  - 不同的Pod，运行在不同的Network Namespace中，拥有独立的网络协议栈。Pod之间只能通过容器开放的端口进行通信，不能通过其他方式进行访问。
- 数据隔离
  - 容器之间利用Namespace进行隔离，在第2章中我们已经对不同的Namespace进行了详细描述。不同Pod的容器，运行在不同的MNT、UTS、PID、IPC Namespace上，相互之间无法访问对方的文件系统、进程、IPC等信息；同一个Pod的容器，其mnt、PID Namespace也不共享。



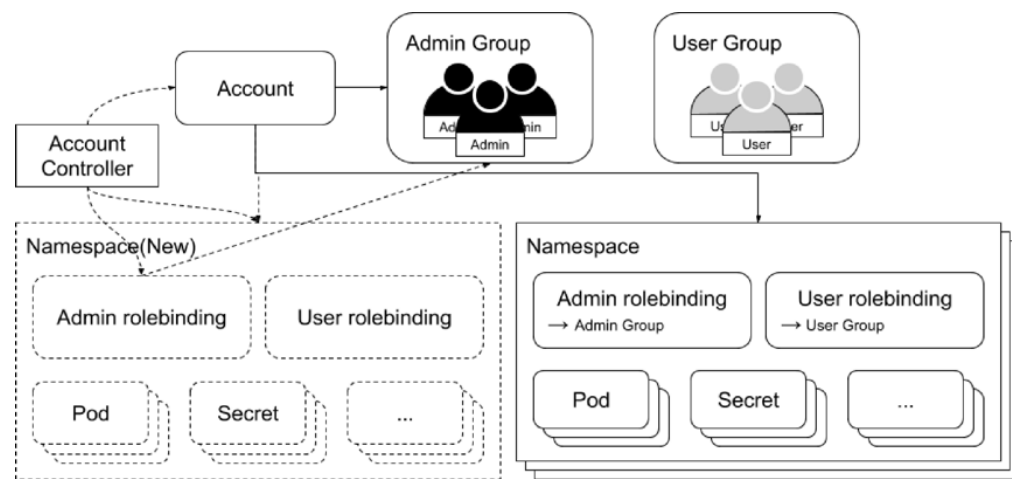
# 租户隔离手段

- Namespace: Namespace属于且仅属于一个租户。
- 权限定义: 定义内容包括命名空间中的Role与RoleBinding。这些资源表示目前租户在归属于自己的命名空间中定义了什么权限、授权给了哪些租户的成员。
- Pod安全策略: 特殊权限指集群级的特定资源定义——PodSecurityPolicy。它定义了一系列工作负载与基础设施之间、工作负载与工作负载之间的关联关系, 并通过命名空间的RoleBinding完成授权。
- 网络策略: 基础设施层面为保障租户网络的隔离机制提供了一系列默认策略, 以及租户自己定制的用于租户应用彼此访问的策略。
- Pod、Service、PersistentVolumeClaim等命名空间资源: 这些定义表示租户的应用落地到Kubernetes中的实体。



# 权限隔离

- 基于Namespace的权限隔离
  - 创建一个namespace-admin ClusterRole，拥有所有对象的所有权限
  - 为用户开辟新namespace，并在该namespace创建rolebinding绑定namespace-admin ClusterRole，用户即可拥有当前namespace所有对象操作权限
- 自动化解决方案
  - 当Namespace创建时，通过mutatingwebhook将namespace变形，将用户信息记录至namespace annotation
  - 创建一个控制器，监控namespace，创建rolebinding为该用户绑定namespace-admin的权限



# Quota管理

- 开启ResourceQuota准入插件。
- 在用户namespace创建ResourceQuota对象进行限额配置。

apiVersion: v1

kind: ResourceQuota

metadata:

name: high-qos-limit-requests

spec:

hard:

limits.cpu: 8

limits.memory: 24Gi

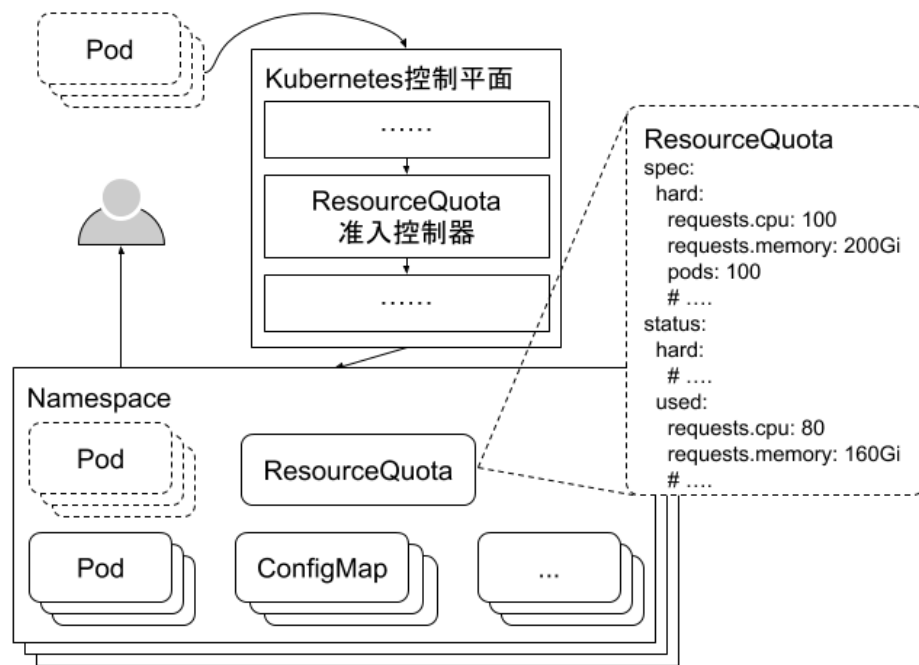
pods: 10

requests.cpu: 4

requests.memory: 12Gi

scopes:

- NotBestEffort



# 节点资源隔离

通过为节点设置不同taint来识别不同租户的计算资源。

不同租户在创建Pod时，增加Toleration关键字，确保其能调度至某个taint的节点。

## 9.2 基于Cluster API搭建一个集群

按照教程通过cluster api 搭建一个集群

# THANKS

 极客时间 | 训练营